

PhishRadar

Détection temps réel de domaines de phishing via les Certificate
Transparency logs

Spécification technique — Plateforme Snowflake
Projet portfolio · Threat Intelligence · Banques françaises

MooM · Data Engineer · Juillet 2026
v1.0 — données 100 % réelles, zéro donnée fictive

1. Contexte et objectifs

Chaque certificat HTTPS émis dans le monde est inscrit, par obligation, dans des journaux publics : les **Certificate Transparency logs** (CT logs). Des milliers de certificats y apparaissent chaque minute. Les cybercriminels y laissent une trace involontaire : avant une campagne de phishing, ils enregistrent un domaine imitant une marque (*bnpparibas-secure.net*, *paypa1.com*) et lui attachent un certificat — visible dans les CT logs des heures, parfois des jours, avant l'attaque.

PhishRadar exploite cette fenêtre : ingestion continue du flux CT, comparaison de chaque nouveau domaine aux marques protégées par matching flou, production d'alertes scorées pour un analyste SOC, et partage sécurisé des indicateurs de compromission (IOC) à des partenaires.

Objectifs du projet

- Démontrer une plateforme Snowflake de bout en bout sur un **vrai flux temps réel** (pas de batch simulé).
- Couvrir l'intégralité des domaines de la certification SnowPro Core, en particulier **Security/RBAC** et **Data Protection & Sharing**, natifs au cas d'usage threat intel.
- Parler à la cible marché (banque, assurance, ESN françaises) : détection du phishing visant BNP Paribas, Crédit Mutuel, Société Générale, etc.
- Réutiliser le savoir-faire de matching flou (distance d'édition, Jaro-Winkler) transposable au MDM.

2. Sources de données (réelles)

Flux CT — interrogé à la source, zéro infrastructure. Le serveur public Calidog (certstream.calidog.io) est hors service depuis mars 2025. Plutôt que d'héberger un agrégateur, le collecteur Python interroge **directement l'API HTTP publique des CT logs** (RFC 6962 : get-sth / get-entries) — Google Argon/Xenon, Let's Encrypt, DigiCert, Sectigo... Les logs actifs sont découverts automatiquement via la liste officielle Chrome (log_list v3). Aucun serveur, aucun Docker, aucun websocket : un seul script (*ingest/ct_poller.py*) qui parse les certificats X.509 et mémorise sa position dans chaque log.

Référentiel marques. Domaines officiels publics de 10 marques : BNP Paribas, Crédit Agricole, Société Générale, Crédit Mutuel, La Banque Postale, Caisse d'Épargne, Banque Populaire, BoursoBank, Hello bank!, PayPal (config/brands.yml).

Message produit par le collecteur (format certstream, champs utilisés)

```
{ "message_type": "certificate_update",
  "data": {
    "leaf_cert": { "all_domains": ["login-bnpparibas.example.com", ...],
                  "sha256": "95868c...", "issuer": {"0": "Let's Encrypt"},
                  "not_before": 1783700888, "not_after": 1791476887 },
    "seen": 1783704398.569,
    "source": { "name": "Google 'Argon2026h2' log" } } }
```

Le collecteur émet le format certstream standard : le SQL aval resterait identique si l'on basculait un jour vers un agrégateur websocket.

3. Architecture

```
CT logs publics (Google Argon/Xenon, Let's Encrypt, ...)
|
| API HTTP RFC 6962 : get-sth / get-entries (polling ~20 s)
v
ct_poller.py (parsing X.509 + pré-filtre marques : ~99,9 % écarté)
| PUT batches JSONL gzip ~qq certs/min
v
===== SNOWFLAKE =====
```

```

@CERT_STAGE --COPY (task 1 min)--> RAW.CERTS (VARIANT)
|
+-- STREAM (CDC) --task--> SILVER.DOMAINS
|   FLATTEN, normalisation, anti-homoglyphes,
|   clustering par date, search optimization
|
+--> GOLD.ALERTS (Dynamic Table, TARGET_LAG 5 min)
|   scoring x marques, types d'attaque, sévérité
|
+--> GOLD.V_TRIAGE (analyste)    GOLD.V_IOC_FEED (secure view)
|
|   SHARE --> reader account partenaire
=====

```

Deux warehouses XS isolent les charges : **WH_INGEST** (COPY, tasks, dynamic table) et **WH_ANALYTICS** (requêtes analystes). Un resource monitor plafonne la consommation à 30 crédits/mois avec suspension automatique.

Choix d'ingestion

V1 : PUT vers stage interne + COPY par task (pattern certif, robuste, sans cloud externe). Évolution « premium » : Snowpipe Streaming SDK pour de l'insertion ligne à ligne — l'architecture SILVER/GOLD reste identique. Le **pré-filtrage côté client** est le contrôle de coût central : seuls les domaines contenant ou ressemblant à une marque (substring ou Levenshtein ≤ 2 sur le label normalisé) sont envoyés à Snowflake — de « tout Internet » à « ce qui touche mes marques ». Le polling direct des CT logs (plutôt qu'un agrégateur hébergé) supprime toute infrastructure : argument d'entretien solide — « je consomme la source primaire, RFC 6962, sans dépendance tierce ».

4. Modèle de données (médailлон)

Couche	Objet	Contenu	Mécanisme
RAW	CERTS	Message certstream complet (VARIANT) + métadonnées	Stockés en fichier COPY par task (1 min), purge auto
RAW	CERTS_STRM	Delta des nouveaux certificats	STREAM (CDC)
SILVER	DOMAINS	1 ligne par (certificat, domaine) : SLD, TLD, format normalisé	Normalisée en SYSTEM\$STREAM\$DATA ; FLUSH
GOLD	BRANDS / COMBO_TERMS	Référentiel marques (token de matching + whitelist)	Tables terminées de combosquatting
GOLD	ALERTS	Alertes scorées : domaine × marque, type d'attaque, score	Quelques TABLES, SEQUEL ET_LAG 5 min
GOLD	V_TRIAGE / V_IOC_FEED	Feed de triage analyste / feed IOC minimal pour machines	TABLES

5. Moteur de détection et scoring

Chaque domaine SILVER est croisé avec chaque marque (CROSS JOIN sur ~10 marques : volume maîtrisé grâce au pré-filtrage). La normalisation anti-homoglyphes (0→o, 1→l, 3→e, 4→a, 5→s, 7→t, 8→b, m→n) est appliquée avant les mesures de similarité.

Type d'attaque	Exemple	Détection SQL
TYPOSQUATTING	bnpparibaz.com, gogle.com	EDITDISTANCE(sld_norm, token) entre 1 et 2
COMBOSQUATTING	bnpparibas-secure.net	CONTAINS(marque) ET terme rassurant (secure, login, connexion...)
HOMOGLYPH	paypa1.com, rnicrosoft.com	sld_norm ≠ sld brut ET distance ≤ 1 après normalisation
BRAND_ABUSE	login.bnpparibas.evil.io	Marque en sous-chaîne du domaine complet

Score de menace (0-100)

Signal	Points
Marque contenue dans le domaine	+50
Terme de combosquatting présent	+25
Distance d'édition 1 / 2 (typosquatting)	+45 / +30
Homoglyphes détectés (normalisation change le label)	+30
Jaro-Winkler ≥ 0,90	+15

Seuil d'alerte : **score ≥ 60**. Sévérité : CRITICAL ≥ 85, HIGH ≥ 70, sinon MEDIUM. Les domaines légitimes des marques (whitelist du référentiel) sont exclus. Philosophie assumée : on ne bloque pas, on **score et on priorise** — comme un vrai outil de threat intel ; les faux positifs se traitent en ajustant seuils et whitelist, décision documentée dans le README.

6. Gouvernance, sécurité, partage

Modèle RBAC (hiérarchie SOC)

Rôle	Périmètre
PR_ADMIN	Possède la base, gère politiques et tags
PR_ENGINEER	Pipeline : RAW/SILVER/GOLD, tasks, warehouse ingestion (rôle du script Python)
PR_ANALYST	Lecture GOLD uniquement (jamais RAW), warehouse analytique, voit les hash de certificats
PR_PARTNER	Uniquement la SECURE VIEW V_IOC_FEED ; lignes CRITICAL/HIGH ; hash masqués

Protection des données

- **Masking policy** sur CERT_SHA256 : visible pour les analystes, « ***MASKED*** » sinon.
- **Row access policy** : un partenaire ne voit que les alertes CRITICAL/HIGH (l'interne voit tout).
- **Tags** : classification TLP (Traffic Light Protocol), standard de la threat intel — GOLD.ALERTS = TLP:AMBER.
- **Secure view** V_IOC_FEED : colonnes minimales, définition masquée.
- **Secure Data Share + reader account** : diffusion du feed IOC à un « partenaire SOC » sans copie de données — démo : connexion au reader account montrant que seules lignes/colonnes autorisées apparaissent.
- **Time Travel** (restauration d'ALERTS à H-1) et **zero-copy clone** (env. de test instantané).

7. Maîtrise des coûts (trial 400 \$)

Levier	Effet
Pré-filtrage client (~99,9 % du flux écarté)	Stockage et compute quasi nuls sur RAW
Warehouses XS, AUTO_SUSPEND 60 s	Facturation à la minute réellement utilisée
Tasks conditionnées SYSTEM\$STREAM_HAS_DATA	Zéro crédit quand il n'y a rien à traiter
Dynamic Table TARGET_LAG 5 min	Rafraîchissement incrémental, pas de recalcul complet
Resource monitor 30 crédits/mois	NOTIFY 75 % → SUSPEND 90 % → KILL 100 %

Estimation en fonctionnement continu : < 2 crédits/jour. Le trial couvre largement les 4 semaines du projet ; les captures d'écran + le repo GitHub restent la preuve durable une fois le trial expiré.

8. Roadmap (4 semaines)

Sem.	Jalon	Contenu
1	Fondations	Trial Snowflake ; 00→10 (infra, RBAC, RAW) ; lancement du collecteur ct_poller.py ; premières ingestions ; véri
2	Pipeline	20→30 : SILVER (stream+task), GOLD (BRANDS, dynamic table ALERTS) ; premières vraies alertes ; captures
3	Gouvernance	40→50 : masking, row access, tags TLP, secure view, share + reader account ; démo partenaire
4	Vitrine	README finalisé, screenshots, schéma d'architecture, dashboard Snowsight (alertes/jour, top marques visées)

9. Mapping certification SnowPro Core

Domaine certif	Preuve dans PhishRadar
Data Loading & Transformation	Stage interne, PUT, COPY INTO, file format JSON gzip, VARIANT, FLATTEN, QUALIFY
Data Pipelines	Streams (CDC), tasks planifiées et conditionnées, chaînage, Dynamic Tables (TARGET_LAG)
Security & Access Control	Hiérarchie de 4 rôles, grants sur objets futurs, séparation des warehouses par usage
Data Protection & Sharing	Masking, row access policy, tags, secure views, Secure Data Share, reader account, Time Travel, cl
Performance & Cost	Clustering key, Search Optimization Service, auto-suspend, resource monitor
Architecture	Médaille RAW/SILVER/GOLD, isolation ingestion/analytique, choix batch vs streaming argumenté

10. Pitch et valorisation

« J'ai construit une plateforme Snowflake qui surveille en temps réel les certificats HTTPS émis dans le monde pour détecter, avant le lancement des campagnes, les domaines de phishing ciblant les banques françaises — scoring de similarité, gouvernance RBAC de niveau SOC et partage sécurisé des indicateurs de menace à des partenaires. »

Arguments différenciants : vrai flux temps réel (rare en portfolio) ; sujet cyber × banque aligné sur la cible marché ; gouvernance native et non plaquée ; matching flou transposable au MDM (Semarchy / Informatica) ; maîtrise des coûts documentée — un réflexe d'ingénieur, pas d'étudiant.

Limites assumées (à mentionner en entretien)

Extraction du domaine enregistrable simplifiée en v1 (TLD composés type .co.uk traités approximativement — évolution : UDF Python tldextract). Détection d'homoglyphes limitée au leetspeak ASCII (évolution : alphabets cyrillique/grec via mapping Unicode). Scoring par règles pondérées, volontairement explicable — une piste ML (features + modèle) est une évolution naturelle, pas un prérequis.